



**RSA SecurID Authentication API
Developer's Guide**

Contact Information

RSA Link at <https://community.rsa.com> contains a knowledgebase that answers common questions and provides solutions to known problems, product documentation, community discussions, and case management.

Trademarks

Dell, RSA, the RSA Logo, EMC and other trademarks, are trademarks of Dell Inc. or its subsidiaries. Other trademarks may be trademarks of their respective owners. For a list of RSA trademarks, go to www.emc.com/legal/emc-corporation-trademarks.htm#rsa.

License Agreement

This software and the associated documentation are proprietary and confidential to Dell Inc. or its subsidiaries, are furnished under license, and may be used and copied only in accordance with the terms of such license and with the inclusion of the copyright notice below. This software and the documentation, and any copies thereof, may not be provided or otherwise made available to any other person.

No title to or ownership of the software or documentation or any intellectual property rights thereto is hereby transferred. Any unauthorized use or reproduction of this software and the documentation may be subject to civil and/or criminal liability.

This software is subject to change without notice and should not be construed as a commitment by Dell Inc.

Third-Party Licenses

This product may include software developed by parties other than RSA. The text of the license agreements applicable to third-party software in this product may be viewed on the product documentation page on RSA Link. By using this product, a user of this product agrees to be fully bound by terms of the license agreements.

Note on Encryption Technologies

This product may contain encryption technology. Many countries prohibit or restrict the use, import, or export of encryption technologies, and current use, import, and export regulations should be followed when using, importing or exporting this product.

Distribution

Use, copying, and distribution of any Dell software described in this publication requires an applicable software license.

Dell Inc. believes the information in this publication is accurate as of its publication date. The information is subject to change without notice.

THE INFORMATION IN THIS PUBLICATION IS PROVIDED "AS IS." DELL INC. MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WITH RESPECT TO THE INFORMATION IN THIS PUBLICATION, AND SPECIFICALLY DISCLAIMS IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Copyright © 2015-2018 Dell Inc. or its subsidiaries. All Rights Reserved.

April 2018

Contents

Preface	6
About This Guide	6
Product Documentation	6
OpenAPI References	6
Graphics Key	6
RSA SecurID Access Support and Service	6
Support for RSA Authentication Manager	7
Support for the Cloud Authentication Service and Identity Routers	7
RSA Ready Partner Program	7
Chapter 1: Getting Started with the RSA SecurID Authentication API	8
RSA SecurID Authentication API Overview	9
RSA SecurID Access Services	9
RSA SecurID Authentication API Definition File	9
Authentication Process Flow	9
Processing Results from Initialize and Verify	11
SSL Certificate Requirements	12
REST Endpoint URLs	12
Required Keys for REST Requests	12
RSA Authentication Manager Requirement	13
Cloud Authentication Service Requirement	13
MethodIDs and Attributes	13
SECURID_NEW_PIN and SECURID_SYSTEM_GENERATED_PIN Attributes for RSA Authentication Manager	13
SECURID_NEWPIN Attributes for the Cloud Authentication Service	14
Method Attributes Returned for Initialize	14
Evaluating Assurance Levels and Primary Authentication Status to Return Authentication Methods	14
Assurance Level Evaluation	15
Assurance Level Evaluation Steps	15
Assurance Level Evaluation Examples	16
Scenario: Assurance level does not include the primary authentication method	17
Scenario: Assurance level includes the primary authentication method	17
Chapter 2: Using the RSA SecurID Authentication API	19

Initialize and Verify Calls	20
Initialize Request	20
Initialize Request Example	21
Initialize Request Parameters	22
Unsupported Parameters	23
Initialize with subjectCredentials	24
Requirements for subjectCredential	24
Initialize with clientDetails	25
Requirements for clientDetails	25
Verify Request	26
Verify Request Example	26
Verify Request Parameters	27
Verify Credential Properties	27
Verify Credential.collectedInputs (NameValuePair) Properties	28
Initialize and Verify Response	28
AuthNResponse Properties	28
AuthNResponse credentialValidationResults Properties	29
AuthNResponse challengeMethods and Related Properties	30
AuthNResponse Content Returned By Initialize and Verify	31
AuthNResponse challengeMethods (ChallengeMethods) Properties	31
AuthNResponse challengeMethods.challenges Properties	31
AuthNResponse challengeMethods...requiredMethods Properties	31
AuthNResponse challengeMethods...versions Properties	31
AuthNResponse challengeMethods...prompt Properties	32
Initialize and Verify Attempt Response Codes	32
Check Status Call	35
Check Status Request	35
Response to Verify	35
Explicit Call to Check Status	35
Check Status Example	35
Check Status Parameters	36
Check Status Response	36
AuthNStatusReponse Properties	36

- Cancel Call 38
 - Cancel Request 38
 - Cancel Request Example 38
 - Cancel Parameters 38
- Chapter 3: Sample Clients for the RSA SecurID Authentication API 39**
 - Sample Client Overview 40
 - Software Requirements 40
 - Files and Directories 40
 - Build Sample Clients for RSA Authentication Manager 40

Preface

About This Guide

RSA SecurID Authentication API is a REST API for developers who want to build clients that send authentication requests to RSA SecurID Access, either through the RSA Authentication Manager server, the Cloud Authentication Service, or both.

RSA strongly recommends using the RSA SecurID Authentication API for developing new clients. This document describes how the REST interface JSON properties are used for authentication and provides guidance for specific scenarios. Each section describes a REST endpoint interface, the expected way the interface is called, the server responses the client should be ready to accept, and security aspects the client should be validating.

Product Documentation

For a complete list of RSA SecurID Access documentation, see "RSA SecurID Access Product Documentation" on RSA Link at <https://community.rsa.com/docs/DOC-60094>.

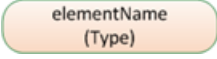
OpenAPI References

The <https://swagger.io> web site contains useful information for developers using REST APIs.

- OpenAPI Specification: <http://swagger.io/specification/>
- OpenAPI Tools: <http://swagger.io/tools/>
- Swagger Editor: <http://swagger.io/swagger-editor/>
- Swagger CodeGen: <http://swagger.io/swagger-codegen/>
- Swagger UI (interactive documentation): <http://swagger.io/swagger-ui/>
- On-Line Support Forums: <http://swagger.io/forum/>

Graphics Key

The symbols in the following table apply to the graphics used in this guide.

Symbol	Description
	Compound element containing one or more elements.
	Simple element of a single type (for example, string, number, and so on).
	Enumerated element and type.
	Groups elements in an array.

RSA SecurID Access Support and Service

You can access community and support information on RSA Link at <https://community.rsa.com>. RSA Link contains a knowledgebase that answers common questions and provides solutions to known problems, product

documentation, community discussions, and case management.

Support for RSA Authentication Manager

Before you call Customer Support for help with the RSA Authentication Manager appliance, have the following information available:

- Access to the RSA Authentication Manager appliance.
- Your license serial number. To find this number, do one of the following:
 - Look at the order confirmation e-mail that you received when you ordered the product. This e-mail contains the license serial number.
 - Log on to the Security Console, and click **License Status**. Click **View Installed License**.
- The appliance software version. This information is located in the top, right corner of the Quick Setup, or you can log on to the Security Console and click **Software Version Information**.

Support for the Cloud Authentication Service and Identity Routers

If your company has deployed identity routers and uses the Cloud Authentication Service, RSA provides you with a unique identifier, called the Customer Support ID, which is required when you register with RSA Customer Support. To see your Customer Support ID, sign in to the Cloud Administration Console and click **My Account > Company Settings**.

RSA Ready Partner Program

The RSA Ready Partner Program website at www.rsaready.com provides information about third-party hardware and software products that have been certified to work with RSA products. The website includes Implementation Guides with step-by-step instructions and other information on how RSA products work with third-party products.

Chapter 1: Getting Started with the RSA SecurID Authentication API

- [RSA SecurID Authentication API Overview 9](#)
- [Authentication Process Flow 9](#)
- [Processing Results from Initialize and Verify11](#)
- [SSL Certificate Requirements 12](#)
- [REST Endpoint URLs 12](#)
- [Required Keys for REST Requests12](#)
- [MethodIDs and Attributes 13](#)
- [Evaluating Assurance Levels and Primary Authentication Status to Return Authentication Methods 14](#)

RSA SecurID Authentication API Overview

The RSA SecurID Authentication API is a REST-based programming interface that allows you to develop clients that process multifactor, multistep authentications through RSA Authentication Manager and the Cloud Authentication Service. The interface definition can be integrated with any programming language.

RSA SecurID Access Services

The Authentication API supports the RSA SecurID Access, described on RSA Link at <https://community.rsa.com/community/products/securid>. This product provides the following services:

- Cloud Authentication Service. Your client must point to this server to support the RSA SecurID Authenticate Tokencode, SecurID Token, Approve, Device Biometrics (such as Fingerprint), SMS Tokencode, Voice Tokencode, and Password authentication methods.
- RSA Authentication Manager. The Authentication Manager server must be deployed in your environment to support RSA SecurID tokens. Authentication Manager supports RSA SecurID Authenticate Tokencodes when configured to integrate with the Cloud Authentication Service. The Authentication API supports Authentication Manager 8.2 Service Pack 1 or later.

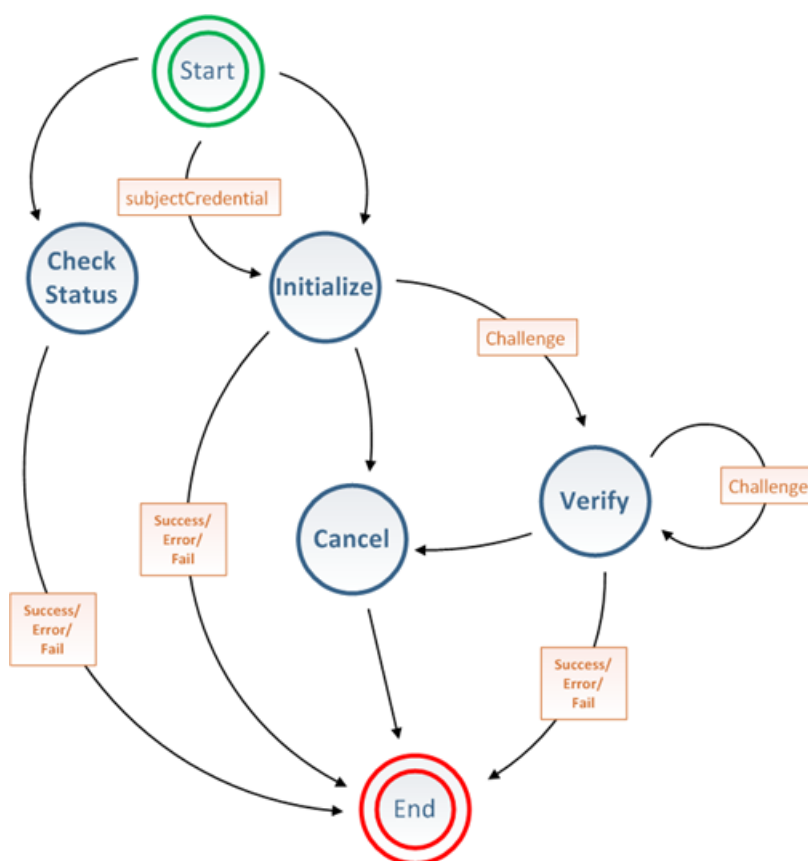
RSA SecurID Authentication API Definition File

RSA provides an OpenAPI interface definition source file containing details on the REST endpoints and JSON objects. This file is the authoritative source for detailed information on all required and optional parameters. You can download the file, **rsa-securid-authenticate-api.yaml**, from RSA Link at this location: <https://community.rsa.com/docs/DOC-71396>.

Note: Do not modify the **rsa-securid-authenticate-api.yaml** file. Modifying this file causes authentication to fail.

Authentication Process Flow

The following diagram shows the general flow of the interface during authentication.



The following steps describe at a high level the client-server process flow during authentication. These steps apply to both the RSA Authentication Manager server and the Cloud Authentication Service.

1. The client calls the Initialize interface.

Calls to the Authentication Manager server use the user login ID (subjectName).

Calls to the Cloud Authentication Service use email or the SecurID Username specified in the Identity Source configuration in the Cloud Administration Console. For Active Directory, the default is sAMAccountName. For other LDAP vendors, this is a vendor-specific value.

Note: When the client sends Initialize with subjectCredentials, it provides the server with credentials to be verified in advance. When subjectCredentials is not used, the server returns information to the client indicating what credentials are required. subjectCredentials is optional for the Initialize call.

2. The Initialize interface responds with the challenge method options and requirements for the user to complete authentication.
3. The client collects challenge method credentials from the user and sends them to the server in a Verify interface call.
4. The Verify interface responds in one of two ways:
 - If the user's authentication is complete and successful, the user is granted access to the requested resource.

- If authentication is not complete, the Verify interface sends the additional challenge requirements the user needs to complete authentication.

Note: When a client (also called the agent) sends a multistep authentication request to the Authentication Manager server, all steps in the transaction must be completed on the same primary or replica instance.

Note: The RSA SecurID Authentication API does not support user password change or device registration as part of the authentication workflow. Users must perform device registration and manage their passwords prior to initiating authentication with clients that use the API.

Processing Results from Initialize and Verify

The Initialize and Verify interfaces return an AuthNResponse JSON object. The challengeMethods element in the response is an array of one or more data elements the server requires to continue or complete an authentication request. In many cases, this array contains a single item, but authentication clients should be able to handle the return of multiple challenge methods and present those methods to the user.

The API supports the following multifactor authentication and mobile methods:

- RSA SecurID hardware and software tokens
- RSA SecurID Authenticate Tokencode
- Approve
- Device Biometrics (for example, Fingerprint)
- SMS Tokencode
- Voice Tokencode
- Password (primary authentication only)

Future server releases might include new challenge methodIds, and clients should be ready to present these new challenge methods in a dynamic manner.

A challenge method may or may not require the user to enter data, as described in the following table.

Data Required?	Result
Yes	The user enters a value which is sent to the server. For example, a password or a new SecurID PIN.
No	The user confirms an authentication event for which no user response data is needed. For example, the user confirms an Approve notification on his phone, or the user acknowledges that he has memorized the system-generated PIN.

For elements that require the user to provide data, these mechanisms can help the client identify how the data should be handled:

- Value Defined. The user enters a value that he will need to remember at a later time, such as a PIN or a password. It is expected that clients require the user to enter the data twice and verify that both entries match.
- Sensitive Data. When the user enters a sensitive value, the data entry should be masked or hidden if possible.

The AuthenticationMethod and MethodPrompt elements contain the information the client can use to generate a prompt for the authentication data and to perform some lightweight verification of the user's entry. It contains

hints such as the minimum and maximum length of the expected data, the prompt that should be used for the data, and the default value, if applicable.

SSL Certificate Requirements

It is important that the client sends authentication requests only to trusted servers. RSA recommends that you configure the client to verify the server's identity by validating that the SSL certificate presented by the server has been issued by a root CA certificate, trusted by the client, directly or indirectly through one or more intermediate CA certificates in the certificate chain.

RSA Authentication Manager generates a root CA certificate and SSL server certificates for each primary and replica instance (server). The root CA is named **RSA root CA for <primary-server-hostname>**. Your company can replace the console certificates with certificates signed by a different CA.

For the Cloud Authentication Service, use your browser to visit the Authentication API REST URL link, where you can view and retrieve the root CA certificate that issued the certificate presented for that link.

REST Endpoint URLs

The Authentication API supports the following REST endpoint interfaces.

REST Endpoint URL	Purpose
/mfa/v1_1/authn/initialize	Initialize is the first call the client sends to the server to start the authentication process.
/mfa/v1_1/authn/verify	After Initialize succeeds, the server returns at least one challenge method. Use Verify to provide authentication credentials, such as an RSA SecurID passcode, to the server.
/mfa/v1_1/authn/status	Checks the status of the authentication attempt.
/mfa/v1_1/authn/cancel	Explicitly cancels an initialized authentication attempt.
/mfa/v1_1/authn/resources	For the RSA Authentication Manager server, this endpoint provides I18N language resources. Use to GET prompt text values for all prompts or for a specific prompt. Not supported by the Cloud Authentication Service.

Note: SSL is required to access the endpoint URLs.

RSA Authentication Manager uses the default port 5555. For example: `https://<fqhn>:5555/mfa/v1_1/authn/initialize`. The Cloud Authentication Service uses the default port 443.

For the Cloud Authentication Service, add `customerid.securid.com/` to each URL. For example, `customerid.securid.com/mfa/v1_1/authn/initialize`, where `customerid.securid.com` is the Authentication Service Domain for your deployment. Your Super Admin can find the Authentication Service Domain in the Cloud Administration Console under **Platform > Identity Routers** on the Registration tab of the Identity Router wizard.

Required Keys for REST Requests

Every call from the client requires a key to securely identify the authentication request. In the **rsa-securid-**

authenticate-api.yaml file this is the client-key. By default, the client-key is included in the HTTP header of authentication requests. Your Super Admin needs to generate this key and send it to you securely.

RSA Authentication Manager Requirement

In RSA Authentication Manager, the client-key is called the Access Key. Your RSA Authentication Manager Super Admin generates an Access Key when enabling the REST API interface using the Security Console.

If the Super Admin enables Hash-Based Message Authentication Code (HMAC) mode, the HTTP header value is an HMAC calculated from the Access Key, a hash of the request body, the Access ID, and other request-specific information. HMAC requires both the Access Key and Access ID. For more information see "Configure the RSA Security Authentication API for Authentication Agents" in RSA Link.

Cloud Authentication Service Requirement

In the Cloud Authentication Service, the client_key is called the Authentication API key, which a Super Admin generates using the Cloud Administration Console. For more information, see "Add an RSA SecurID Authentication API Key" in RSA Link.

MethodIDs and Attributes

The following table lists the supported MethodIDs for the RSA SecurID Authentication API.

Note: In this table, AM refers to RSA Authentication Manager and CAS refers to the Cloud Authentication Service.

MethodID/Attribute Name	Description	AM	CAS
SECURID	SecurID passcode or Authenticate Tokencode	X	X
SECURID_NEXT_TOKENCODE	SecurID tokencode (PIN not required)	X	
SECURID_NEXT_CODE	SecurID tokencode (PIN not required). The client sends this attribute in Credential.collectedInputs. The value is the user's next passcode.		X
SECURID_NEWPIN	SecurID Personal Identification Number (PIN)	X	X
SECURID_SYSTEM_GENERATED_PIN	System-generated PIN. No value required.	X	
PASSWORD	User's password for primary authentication only. Not used in access policies.		X
TOKEN	RSA SecurID Authenticate Tokencode. Requires the user to have a registered device.		X
APPROVE FINGERPRINT	Methods that require the user to have a registered device. FINGERPRINT may include additional biometric methods that require a registered device.		X
SMS	Requires the user to have a phone that can receive text messages.		X
VOICE	Requires the user to have a phone that can receive voice calls.		X

SECURID_NEW_PIN and SECURID_SYSTEM_GENERATED_PIN Attributes for RSA Authentication Manager

These method attributes support clients using the RSA Authentication Manager legacy ACM_RQPIN_MSG... structure from the C legacy UDP protocol header file. The server provides these attributes when a new PIN is

required or a system-generated PIN is provided.

Method Attribute Name	Type	Value
IS_SYSTEM_GENERATED_TYPE	BOOLEAN	"true" or "false." If "true," the system generates the PIN and the valueRequired field is "false."
SYSTEM_GENERATED_PIN	STRING	System-generated PIN. This PIN may also be provided in the promptArg array.
PIN_REQUIREMENT	STRING (Enumerated)	REQUIRE_ALPHANUM, REQUIRE_ALPHABETIC_ONLY, and REQUIRE_NUMERIC_ONLY. Defines the PIN alphanumeric character requirements.
PIN_ALPHABETIC_CHAR_COUNT	INT32	Number of alphabetic characters required.
PIN_NUMERIC_CHAR_COUNT	INT32	Number of numeric characters required.

SECURID_NEWPIN Attributes for the Cloud Authentication Service

Method Attribute Name	Type	Value
pinGeneration	STRING	USER_DEFINED or SYSTEM_GENERATED
pinFormat	STRING	NUMERIC or ALPHANUMERIC
maxPinLen	STRING	Maximum length, for example, "8"
minPinLen	STRING	Minimum length, for example, "4"
systemPin	STRING	System-generated PIN
newPinPrompt	STRING	Sends the PIN policy from server to client, for example, "newPinPrompt: [maxPinLen:8, minPinLen:4, pinFormat:ALPHANUMERIC, pinGeneration:USER_DEFINED]". This applies only when SECURID_NEW_PIN is the next expected method (with a priority of "1").

Method Attributes Returned for Initialize

The Cloud Authentication Service returns the following method attribute after the Initialize call.

Attribute Name	Attribute Value	Data Type	Direction	Description
METHOD_NOT_APPLICABLE	DEVICE__NOT__REGISTERED DEVICE__NOT__CAPABLE METHOD__NOT__ENROLLED	String	Output	Method cannot be verified until the user registers a device or method, for example, Approve.

Evaluating Assurance Levels and Primary Authentication Status to Return Authentication Methods

After the Cloud Authentication Service receives an Initialize request from an authentication client, the server

determines which authentication methods to return to the client by evaluating the following factors:

- Assurance level (specified in a policy or by itself). See [Assurance Level Evaluation below](#) for a description of this process.
- Verification of the primary authentication method. Users are challenged for primary authentication (for example, username and password) when they initially attempt to access the application. After primary authentication, the assurance level determines if additional authentication is required using RSA SecurID, RSA SecurID Authenticate Tokencode, Approve, Device Biometrics, SMS Tokencode, or Voice Tokencode.

The client might not require primary authentication. If it is required, the server might return different results depending on whether primary verification succeeds.

- If the client's required assurance level or higher contains the primary authentication method in the list of methods required for additional authentication, and if that method is satisfied during primary authentication, the server does not return that method to the client.

For examples of server behavior, see [Scenario: Assurance level does not include the primary authentication method on page 17](#) and [Scenario: Assurance level includes the primary authentication method on page 17](#).

Assurance Level Evaluation

When the Cloud Authentication Service receives an authentication request, it evaluates the client's assurance level to determine which challenge (authentication) methods to return to the client. Clients that are configured as relying parties in the Cloud Administration Console are assigned an access policy in the relying party settings. If an Initialize request specifies the assurance level or an access policy, that request overrides the configuration specified in the Cloud Administration Console.

Assurance levels define the authentication methods required to access applications. RSA SecurID Access provides three assurance levels: High, Medium, and Low. Each level indicates the relative strength and security of the authentication methods within that level. The Super Admin configures authentication options for each assurance level. An option can be a standalone authentication method such as Device Biometrics, or it can combine two methods connected by AND operators, such as SecurID Token and Approve.

The first option listed for an assurance level on the Assurance Levels page is presented as the default for each new user when he or she authenticates to an application or client assigned to that assurance level for the first time. A user can select another option at any time, as long as the assigned assurance level or a higher assurance level contains additional options that the user can complete. When a user successfully authenticates with an option, that option becomes the user's default for future authentications for that assurance level.

For more information on assurance levels, see "Assurance Levels" on RSA Link at <https://community.rsa.com/docs/DOC-53965>.

Assurance Level Evaluation Steps

When the Cloud Authentication Service receive an authentication request, it performs the following evaluation:

1. The server checks the assurance level assigned to the client.
2. The server creates a list containing the challenge methods for the assurance level and the methods for all higher assurance levels.
3. The server orders the challenge methods according to the assurance level configuration in the Cloud Administration Console, from lowest to highest, and moves the default method to the top if available.
4. If any duplicate methods exist, the server eliminates the duplicates.

5. If primary authentication is not required, the server returns the remaining methods to the client. If primary authentication is required, the server verifies the primary authentication method, then determines which methods to return to the client as shown in the following scenarios.

Assurance Level Evaluation Examples

The following table provides examples of expected server behavior during authentication when the default option is available in the assigned assurance level or a higher level. The default option is moved to the top of the returned list of challenge methods.

Assurance Level	Methods Defined for That Level	Default (Last Successful Authentication Method)	Methods Returned to the Client Plus Higher Levels
HIGH	(SECURID AND APPROVE) OR (SECURID AND DEVICE BIOMETRICS)	(SECURID AND DEVICE BIOMETRICS)	(SECURID AND DEVICE BIOMETRICS) OR (SECURID AND APPROVE)
MEDIUM	(DEVICE BIOMETRICS) OR (SMS TOKENCODE)	(SMS TOKENCODE)	(SMS TOKENCODE) OR (DEVICE BIOMETRICS) OR (SECURID AND APPROVE)
MEDIUM	(DEVICE BIOMETRICS) OR (SMS TOKENCODE)	(SECURID AND APPROVE)	(SECURID AND APPROVE) OR (DEVICE BIOMETRICS) OR (SMS TOKENCODE)
LOW	(APPROVE) OR (AUTHENTICATE TOKENCODE)	(AUTHENTICATE TOKENCODE)	(AUTHENTICATE TOKENCODE) OR (APPROVE) OR (DEVICE BIOMETRICS) OR (SMS TOKENCODE)
LOW	(APPROVE) OR (AUTHENTICATE TOKENCODE)	(DEVICE BIOMETRICS)	(DEVICE BIOMETRICS) OR (APPROVE) OR (AUTHENTICATE TOKENCODE) OR (SMS TOKENCODE)

The following table provides examples of expected server behavior during authentication when the default option is not available.

Assurance Level	Methods Defined for That Level	Methods Returned to the Client Plus Higher Levels
HIGH	(SECURID AND APPROVE) OR (SECURID AND DEVICE BIOMETRICS)	(SECURID AND APPROVE) OR (SECURID AND DEVICE BIOMETRICS)
MEDIUM	(DEVICE BIOMETRICS) OR (SMS TOKENCODE)	(DEVICE BIOMETRICS) OR (SMS TOKENCODE) OR (SECURID AND APPROVE)

Assurance Level	Methods Defined for That Level	Methods Returned to the Client Plus Higher Levels
LOW	(APPROVE)	(APPROVE) OR (DEVICE BIOMETRICS) OR (SMS TOKENCODE)

If the method list contains duplicates, the server removes the duplicate sets, as shown in the following example.

Original Methods List Containing Duplicates	Methods List After Duplicate Removal
(SECURID AND TOKEN) OR (SECURID AND SMS TOKENCODE) OR (TOKEN)	(SECURID AND SMS TOKENCODE) OR (TOKEN)
(SECURID AND APPROVE) OR (SECURID AND DEVICE BIOMETRICS) OR (APPROVE) OR (DEVICE BIOMETRICS)	(APPROVE) OR (DEVICE BIOMETRICS)

If a method requires a device and the user has not registered a device, the method is included in the methods list marked with the METHOD_NOT_APPLICABLE attribute and the value DEVICE_NOT_REGISTERED. The user must register a device before using this method.

Scenario: Assurance level does not include the primary authentication method

When the client's assurance level does not include the primary authentication method in the list of methods used for additional authentication, the following behavior occurs:

- If primary method verification is not required, or if it is required and authentication succeeds, the server returns only the methods from the required assurance level or higher. It does not return the primary method.
- If primary method verification is required and fails, the server adds the primary method to the list of methods in the assurance level and returns all methods to the client.

The following examples illustrate this behavior. In the following table, the assurance level specifies (DEVICE BIOMETRICS) OR (SECURID AND APPROVE).

Server Action	Server Returns These Access Policy Methods to Client	Explanation
No primary authentication.	(DEVICE BIOMETRICS) OR (SECURID AND APPROVE)	Returns only methods from the required assurance level or higher.
Password is successfully verified as primary method.	(DEVICE BIOMETRICS) OR (SECURID AND APPROVE)	Returns only methods from the required assurance level or higher.
Unsuccessful verification of Password as primary method.	(PASSWORD AND DEVICE BIOMETRICS) OR (PASSWORD AND SECURID AND APPROVE)	The server adds Password to the list of methods from the required assurance level.

Scenario: Assurance level includes the primary authentication method

When the client's assurance level includes the primary authentication method in the list of methods required for additional authentication, the following behavior occurs:

1. The server attempts to validate the credential received from the client for primary authentication.

2. If validation succeeds, the server removes the primary authentication method from the list of methods in the required assurance level or higher and does not return that method to the client. If validation fails, the server adds the primary method to the list of methods in the assurance level.
3. The server eliminates any redundant methods and returns the remaining methods to the client, which are required to satisfy the assurance level.

The following example illustrates this behavior. In the following table, the assurance level specifies (SECURID AND APPROVE) OR (SMS TOKENCODE).

Server Action	Server Returns These Access Policy Methods to Client	Explanation
No primary authentication.	(SECURID AND APPROVE) OR (SMS TOKENCODE)	Returns only methods from the required assurance level or higher.
Successful verification of SECURID as primary method	(APPROVE) OR (SMS TOKENCODE)	SECURID is counted as being completed and is removed from the list of methods from the required assurance level or higher and the remaining methods are sent to the client.
Unsuccessful verification of SECURID as primary method	(SECURID AND APPROVE) OR (SECURID AND SMS TOKENCODE)	SECURID is added to the list of methods from the required assurance level or higher and that list is returned to the client.

Chapter 2: Using the RSA SecurID Authentication API

- Initialize and Verify Calls 20
- Check Status Call 35
- Cancel Call 38

Initialize and Verify Calls

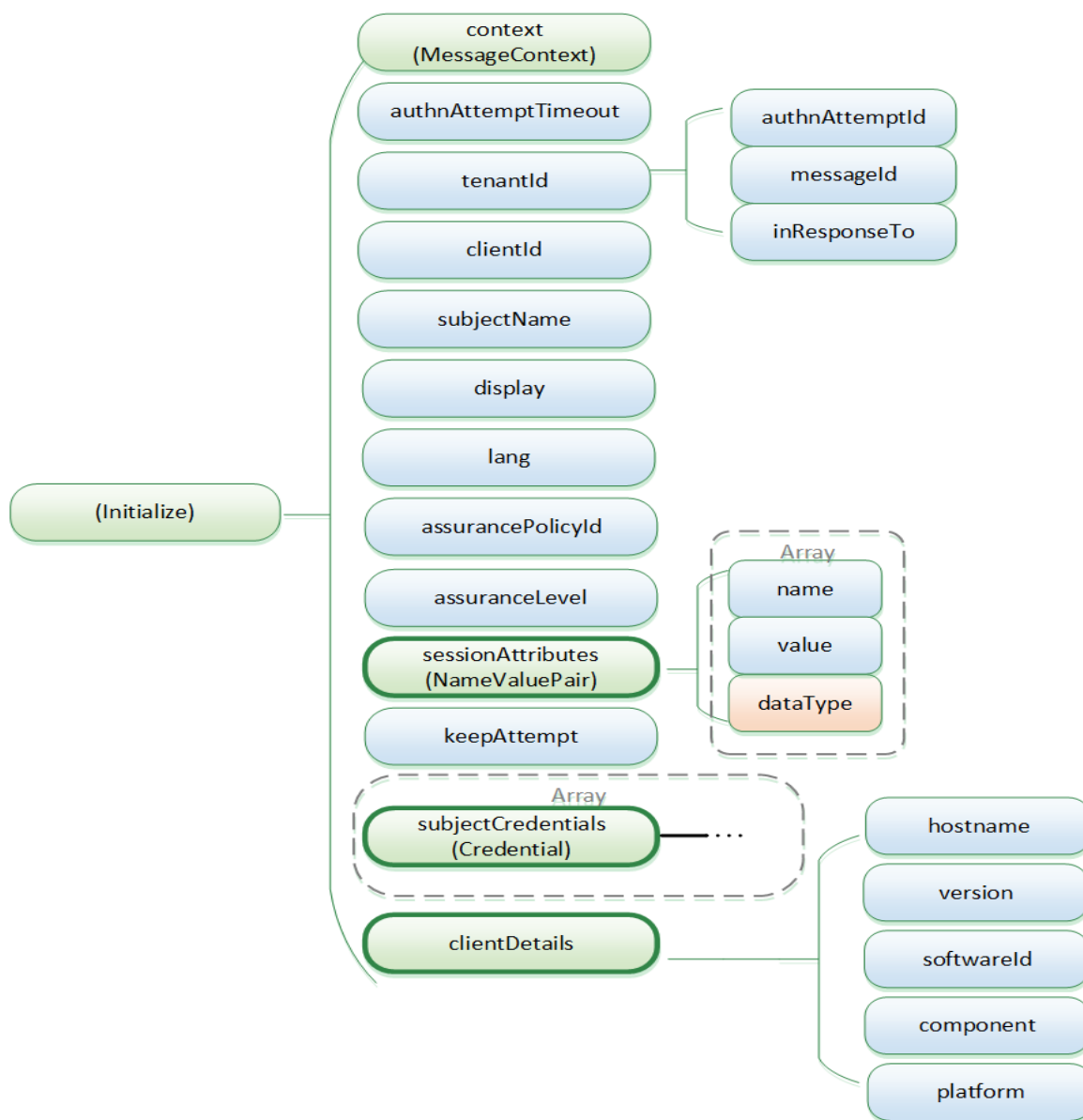
Initialize Request

The Initialize request is the first call to the server. This call starts the authentication process.

Note: Make sure you obtain the necessary keys from your Super Admin. For details, see [Required Keys for REST Requests on page 12](#).

When you use Initialize without subjectCredentials, the server response tells the client which credentials are required. When you use Initialize with subjectCredentials, the Initialize request provides the server with credentials to be verified in advance. For more information, see [Initialize with subjectCredentials on page 24](#).

The following graphic illustrates the relationships among the Initialize properties.



Initialize Request Example

The following snippet shows a sample initialize request in Java. The client provides values for the variables in *italics*.

Note: The client-key value is hard-coded in the **rsa-securid-authenticate-api.yaml** file.

```

ApiKeyAuth client-key = (ApiKeyAuth)mfaApi.getApiClient
().getAuthentication("client-key");

client-key.setApiKey(<api_key_from SuperAdmin>); // enter a valid api-key

MessageContext msgContext = new MessageContext();
  
```

```

msgContext.setMessageId(<random_message_id>); // new message id generated
by the client

Initialize initRequest = new Initialize();

initRequest.setContext(msgContext);

initRequest.setSubjectName(<user_name>); // user's email or sAMAccountName

initRequest.setClientId(<client_id>); //client's unique identifier

initRequest.setAssurancePolicyId(<client_policy_name>); // policy name

AuthNResponse serverResponse = mfaApi.initialize(initRequest);

```

Initialize Request Parameters

The Initialize request requires the following parameters.

Parameter	Description	Type	Required?
clientId	Determines principal identity and access control. Value for Authentication Manager: Agent name Value for Cloud Authentication Service: A string that will be displayed in the mobile notifications in the User Event Monitor in the Cloud Administration Console.	String	Required for Authentication Manager. Optional for the Cloud Authentication Service.
subjectName	Identifies the user account requesting the authentication. Value for Authentication Manager: User Login uid or alias. Value for Cloud Authentication Service: Email or must match the SecurID Username specified in the Identity Source configuration in the Cloud Administration Console. For Active Directory, this is sAMAccountName. For other LDAP vendors, this is a uid.	String	Required
assurancePolicyId	Access policy name configured in the Cloud Administration Console. Obtain this name from your Cloud Authentication Service Super Admin.	String	Optional for the Cloud Authentication Service. Must specify this or assuranceLevel in request.
assuranceLevel	Required minimum assurance level for authentication: LOW, MEDIUM, or HIGH Authentication methods available for each assurance level are specified in the Cloud Administration Console.	String	Optional for the Cloud Authentication Service.

Parameter	Description	Type	Required?
			Must specify this or assurancePolicyId in request. You might use this parameter if you do not want to create policies and want all users to perform additional authentication.
Context	Common authentication request context data.	Object	Required
keepAttempt	"False" (default) - Remove completed or canceled authentication attempts from the server. "True" - Retain completed or canceled authentication attempts until they are removed or expired. Set to "true" if a subsequent CheckStatus call will be made. A completed authentication attempt is any attempt for which an Initialize or Verify call has returned a ResponseCode other than CHALLENGE or IN_PROCESS.	Boolean	Required for the Cloud Authentication Service. Not implemented for Authentication Manager. The attemptID is always removed from the server.
clientDetails	Contains client details.	Object	Optional for Authentication Manager and the Cloud Authentication Service.

Note: Do not use the Credential field versionId (string). It is ignored.

Unsupported Parameters

Do not include authnAttemptId with the Initialize call for either server. If provided, the server ignores it and returns a new, random authnAttemptId in the AuthNResponse context property.

The following table lists unsupported parameters for each server. These parameters might be supported in future versions of the RSA SecurID Authentication API.

Parameter	Authentication Manager 8.2 SP1 or Later	Cloud Authentication Service
authnAttemptTimeout (number)	Not supported	Supported
lang (string)	Not supported	Not supported
sessionAttributes (NameValuePair array)	Not supported	Supported
tenantId (string)	Not supported	Not supported

Parameter	Authentication Manager 8.2 SP1 or Later	Cloud Authentication Service
display (string)	Not supported	Not supported
assurancePolicyId (string)	Not supported	Supported
assuranceLevel (string)	Not supported	Supported
authSchema (string)	Not supported	Supported

Initialize with subjectCredentials

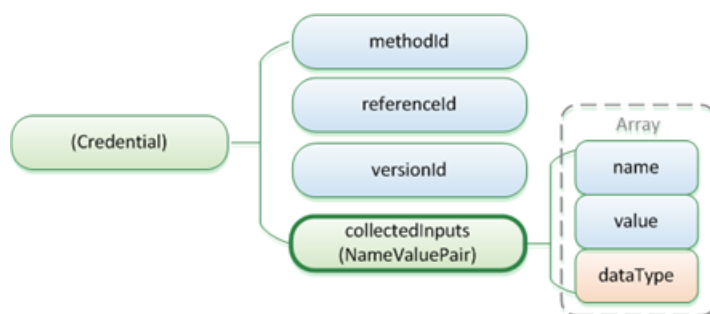
When the client sends Initialize with subjectCredentials, it provides the server with credentials to be verified in advance. When the client sends Initialize without subjectCredentials, the server returns information to the client indicating what credentials are required. subjectCredentials is optional for the Initialize call.

The following table describes how Initialize and subjectcredentials works when sent to the Cloud Authentication Service.

Client Request to the Cloud Authentication Service	Results
Initialize without subjectCredentials	Primary authentication is not performed or expected. The server requests the client to provide the authentication methods required by the access policy.
Initialize with subjectCredentials	The request includes the primary authentication credentials and the credentials required by the access policy. If the client provides an incorrect primary authentication credential (for example, wrong password), the server returns a request for the primary method in addition to the methods required by the access policy.

When the client sends Initialize with subjectCredentials to Authentication Manager, the client provides the user's SecurID passcode in advance, before the server can return it as a challenge in a response. The caller can complete a single-step authentication without multiple round-trips to the server. If the user's token is in New PIN mode or requires the next tokencode to complete the authentication, calling Initialize with credentials still results in a CHALLENGE response.

The following graphic shows the relationships among the properties for subjectCredential.



Requirements for subjectCredential

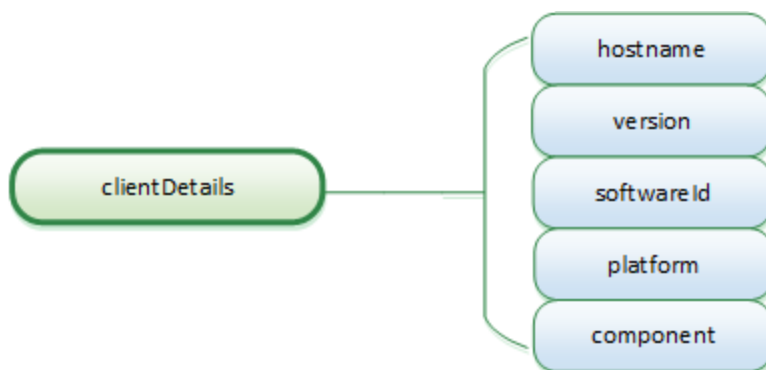
The following table describes requirements for subjectCredential.

Parameter	Description	Type	Required?
methodId	Method to verify. Values for Cloud Authentication Service: PASSWORD SECURID Value for Authentication Manager SECURID	String	Yes
collectedInputs	A NameValue pair. The named methods are a subset of those given in MethodIDs and Attributes on page 13 . For RSA Authentication Manager, the collectedInput name must be the same as the methodId (SECURID). The collectedInputs value is the user's passcode. For the Cloud Authentication Service, only SECURID or PASSWORD are accepted.	Array	Yes

Initialize with clientDetails

Some clients, such as newer RSA authentication agents, can send Initialize with clientDetails. It provides the additional client information to the server for agent reporting purposes. When the client sends Initialize without clientDetails, the server can only report partial information about the client.

The following graphic shows the relationships among the properties for clientDetails.



Requirements for clientDetails

The following table describes requirements for clientDetails.

Parameter	Description	Type	Required?
hostname	Client fully qualified hostname.	String	Optional
version	Version for the installed client.	String	Optional
softwareId	Unique ID generated for each client installation.	String	Optional
platform	The operating system on which the client is installed. The version of the operating system might be included.	String	Optional

Parameter	Description	Type	Required?
component	Installed client name.	String	Optional

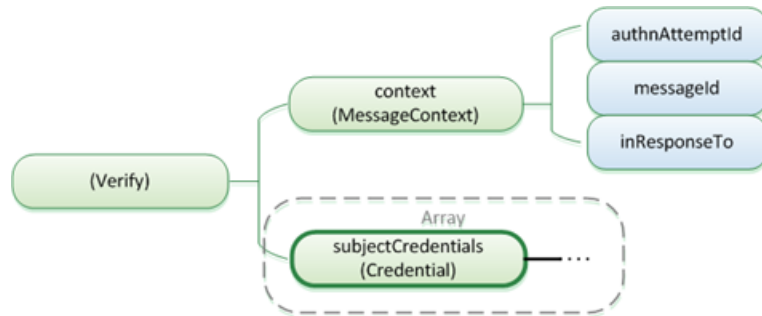
Verify Request

Use the Verify interface to provide credential or confirmation values for challenge methods returned from a previous Initialize or Verify call (authnResponseCode is CHALLENGE).

To invoke the verify call, the client does the following:

1. Sets the authnAttemptId field to the value returned by the server in its initialization response, in the context field of the verification request.
2. Creates a new messageId and sets it in context.
3. Sets the inResponseTo field to the value of messageId returned by the server, in context.
4. For the challenge method returned by the server, collect the user's credentials. Create a credential object with user's credentials in the collectedInputs field and method name in the methodId field. Add the credential object to the subjectCredentials list of the verification request. If the server generated a referenceID for this methodID, include the referenceID.
5. Invokes the verify call with a single credentials.

The following graphic illustrates the relationships among the Verify properties.



Verify Request Example

The following example shows how to invoke verify in Java.

```
// authentication attempt id returned by the server
MessageContext msgContext = new MessageContext();
msgContext.setAuthnAttemptId(serverAuthnAttemptId);

// new message id generated by the client
msgContext.setMessageId(clientNextMessageId);

// message id in the server's last response
msgContext.setInResponseTo(serverLastMessageId);

// methodId of a challenge method returned by the
serverCredentials userCredentials = new Credentials();
NameValuePair cred = new NameValuePair();
cred.setName(expectedAttributeName);

// credential provided by the user
cred.setValue(userInput);
```

```
// methodId of a challenge method returned by the server
userCredentials.addCollectedInputsItem(cred);
userCredentials.setReferenceId(serverProvidedReferenceId)
userCredentials.setMethodId(challengeMethodName);

Verify verifyRequest = New Verify();
verifyRequest.setContext(msgContext);
verifyRequest.addSubjectCredentialsItem (userCredentials);
AuthNResponse serverResponse = mfaApi.verify(verifyRequest);
```

Verify Request Parameters

Property	Description	Type	Required?
subjectCredentials	A list of credentials based on one or more of the previous ChallengeMethods.	Array	Yes
MessageContext	Common authentication request context data sent to and returned by the server. The server response contains the authentication attempt ID, which must be returned in subsequent calls.	Object	Yes

Verify Credential Properties

Property	Description	Type	Required?
methodId	Method ID of the credential corresponding to the collected inputs. Represents a challenge from a previous call. See Initialize and Verify Response on the next page .	String	Yes
versionId	Version of the authentication method corresponding to the collected inputs. This is the version ID, such as "1.0.0," of the authentication method version to be verified. See Initialize and Verify Response on the next page .	String	Yes for the Cloud Authentication Service. Not implemented for Authentication Manager.
referenceId	The server creates and returns a referenceId to the client. Any subsequent verify calls from client to poll the status of the method must contain the referenceId in the verify request. If a subsequent verify call for an IN_PROCESS method does not contain the referenceId, the server assumes it is a new request, cancels the current inprocess method, and initiates a new one, resulting in a new referenceId. From server to client, the server sets the referenceId in the method's AuthenticationMethodVersion.referenceId field. From client to server, the client is expected to set the referenceId in the method's Credential.referenceId field.	String	Yes for the Cloud Authentication Service Not implemented for Authentication Manager.
collectedInputs	NameValuePair objects containing the input obtained from the user that is verified by the server.	Array	Yes

Verify Credential.collectedInputs (NameValuePair) Properties

Property	Description
name	Name must be the same as the credential methodId. This is the method ID of a challenge from a previous call. For more information, see Initialize and Verify Response below .
value	The credential value entered by the end user, if applicable. This is the collected input value for AuthenticationMethodVersion with a valueRequired property of "true." This can be null and any value must be ignored if no value is required.

Initialize and Verify Response

The Initialize and Verify calls return an AuthNResponse. The response indicates if the credentials are valid and if the user must provide additional authentication credentials.

AuthNResponse Properties

AuthNResponse returns the following information.

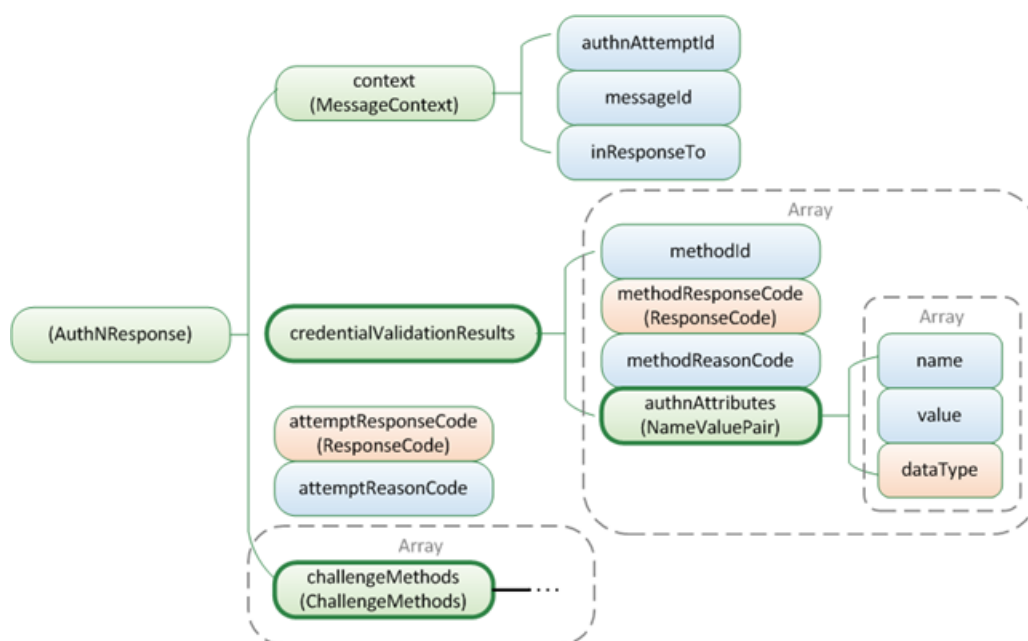
Parameter	Description	Type
credentialValidationResults	Each element of this array corresponds one-to-one with a credential provided in the Initialize or Verify call.	Array
attemptResponseCode	Response status for the entire Initialize call. For example, the credentialValidationResults may indicate success, but this value may be CHALLENGE if additional authentication is required. "SUCCESS" indicates that the request is complete and no further Verify calls are required.	String
attemptReasonCode	A reason code for the entire Initialize request.	String
challengeMethods	If the attemptResponseCode is CHALLENGE, this contains the authentication requirements that must be completed. For a list of challenge methods, see MethodIDs and Attributes on page 13 . If the attemptResponseCode is SUCCESS this parameter is empty.	Object

The following parameters must be returned:

- context (MessageContext)
- challengeMethods (ChallengeMethods)

The server returns the MessageContext.authnAttemptID in the Initialize response. The client must provide this value in each subsequent response. Each server response also contains a random message ID (MessageContext.messageId). RSA recommends that the client validate the MessageContext.inResponseTo property returned by the server as specified in the **rsa-securid-authenticate-api.yaml** file. This value should correspond to the messageId that the client provides in the prior Initialize or Verify call context property.

The following graphic illustrates the relationships among the AuthNResponse properties.



AuthNResponse credentialValidationResults Properties

If the Initialize call contains credentials, the server response must indicate if the credentials are valid. This table describes the content of the credentialValidationResults that can be returned from the Initialize call.

Property	Description	Value
methodId	A method ID to indicate that the credential validation results contain a result from verifying a SecurID passcode authentication.	See MethodIDs and Attributes on page 13
methodResponseCode	SUCCESS - Authentication is completed successfully. FAIL - Unsuccessful authentication. User credentials were incorrect. IN_PROCESS - Authentication is not complete and is in process. For some out of band (PUSH) methods, the client must retry with a Verify call. A Reference ID is returned in the method response. Subsequent verify calls must pass this ID as part of the credential. CHALLENGE - The method is incomplete and method(s) in the challengeMethods are required. For example, the challengeMethods may contain data requiring the client to perform a secondary challenge. ERROR - An error occurred in processing the client request. The authnAttemptId is no longer valid.	SUCCESS FAIL ERROR CHALLENGE IN_PROCESS ERROR
methodReasonCode	Details about the result of the methodResponseCode.	String
authnAttributes	Authentication Manager returns no authentication attributes.	

Property	Description	Value
	The Cloud Authentication Service may return an array of attributes resulting from a successful authentication. Attributes are specific to the authentication type and request. For example, this may contain RADIUS attributes or data to permit additional exchanges such as an offline data download ticket. This will only be optionally provided if the methodResponseCode is "SUCCESS".	

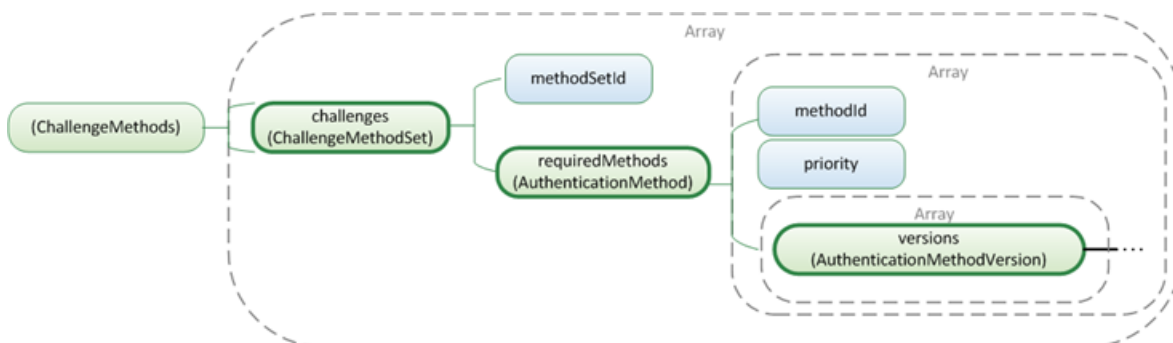
AuthNResponse challengeMethods and Related Properties

The challengeMethods describe information the user must provide or confirm to complete or continue the authentication. The client must use elements from the challengeMethods to generate an authentication user-interface.

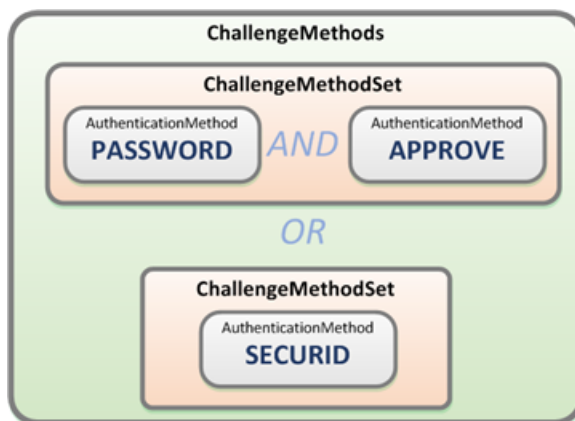
An exception is when the caller provides credentials to the Initialize interface and no further credentials may be needed. In this case, the challengeMethods must be empty. The server may respond with CHALLENGE even when correct credentials are provided with the Initialize call. For example, the user's passcode may be correct, but the next tokencode may be required to complete the authentication.

Challenge methods are described in [MethodIDs and Attributes on page 13](#).

The following graphic shows the relationships among the ChallengeMethods properties.



In the following graphic, ChallengeMethods returns two ChallengeMethodSets, requiring the client to satisfy either two methods (Password and Approve) or one method (SecurID Token), as determined by the assurance level for the access policy being used.



AuthNResponse Content Returned By Initialize and Verify

The following sections describe AuthNResponse content that may be returned from the Initialize and Verify calls. Some section titles use an ellipsis to abbreviate the complete path to the element in the hierarchy. The outermost property contains an array of ChallengeMethodSet objects.

Any one of these challenge methods can be satisfied to complete the authentication.

AuthNResponse challengeMethods (ChallengeMethods) Properties

Property	Description	Values
challenges	Indicates how the user must be challenged to complete authentication.	An array of ChallengeMethodSet containing a single item.

AuthNResponse challengeMethods.challenges Properties

All authentication methods returned in the challengeMethodSet must be satisfied to complete authentication.

Property	Values	Description
requiredMethods	An array of AuthenticationMethod containing a single item.	AuthenticationMethod indicates what data the user must provide to complete authentication.

AuthNResponse challengeMethods...requiredMethods Properties

AuthenticationMethod returns details about data the user must provide to successfully complete authentication.

Property	Values	Description
methodId	See MethodIDs and Attributes on page 13 .	Method IDs associated with the data the user must provide to complete authentication.
priority	Number. The default is 50.	Used by the Cloud Administration Console to indicate to the client the order or priority to use when processing methods. One is the highest priority. Higher numbers indicate lower priority. This value helps ensure that users respond to time-sensitive methods first. For example, SecurID Next Tokencode and New PIN modes have a priority of 1. When a method does not apply the user, it is assigned a priority of 100.
versions	An array of AuthenticationMethodVersion containing a single item.	AuthenticationMethodVersion provides details on the data the user must provide to complete authentication.

The server may return the **priority**, but because the **versions** array contains a single item, the client can ignore this property.

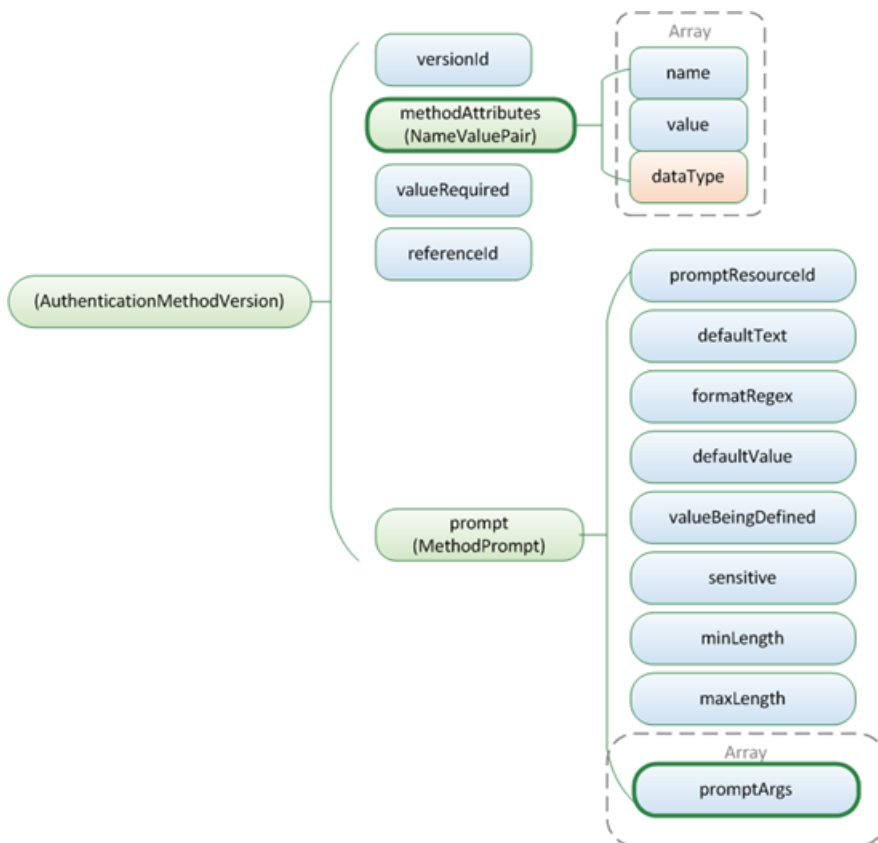
AuthNResponse challengeMethods...versions Properties

AuthenticationMethodVersion returns details about data the user must provide to complete authentication.

Property	Values	Description
versionId	"1.0.0"	The version of the authentication method.
methodAttributes	Array of NameValuePair. See MethodIDs and Attributes on page 13 .	Provided by the server for the SECURID_NEW_PIN or SECURID_SYSTEM_GENERATED_PIN methods.

Property	Values	Description
valueRequired	True or False	True indicates the user must enter a value. This value should be False for methods that do not require the user to enter a value, for example, the Approve method. No value is required if Authentication Manager is generating a SECURID_SYSTEM_GENERATED_PIN.
prompt	MethodPrompt	MethodPrompt provides details on how to prompt the user to obtain the data that must be provided to complete authentication.

The following graphic illustrates the relationships among the AuthenticationMethodsVersion properties.



AuthNResponse challengeMethods...prompt Properties

AuthenticationMethodVersion provides data to the client on how the user should be prompted. The prompt (MethodPrompt) object is returned.

Initialize and Verify Attempt Response Codes

The following table describes attemptResponseCodes and attemptReasonCodes for the Initialize and Verify interfaces. These response codes are provided in two places:

- In the top level of the AuthNResponse for the entire request.
- In the credentialValidationResult for each specific credential.

attemptResponseCode	Description	attemptReasonCode	context	challengeMethods	credentialValidationResults
ERROR	The server cannot handle the initialize request. The request is malformed or invalid.	Possible reasons: VERIFY_MISSING_ATTEMPT_OR_MESSAGE_ID No server cache found for attemptId: <xxx> Corrupt message chain. Got a inResponseToId as: <xxx>, but expected: <yyy>	Undefined	Undefined	Undefined
FAIL	Access denied for this request. The user might have entered the wrong credential.	Possible reasons: ATTEMPT_TIMED_OUT	Populated with IDs.	Contains challenge sets.	Empty or populated with results.
SUCCESS	The authentication request is complete without further challenge.	NO_CHALLENGES_REMAINING	Populated with IDs.	Empty	Empty or populated with results.
CHALLENGE	The authentication request requires further challenge.	Possible reasons: METHOD_VERIFY_FAILED_RETRY METHOD_VERIFY_ERROR_RETRY METHOD_VERIFY_CHALLENGE METHOD_VERIFY_IN_PROCESS	Populated with IDs.	Contains challenge sets.	Empty or populated with results.
IN_PROCESS	The server initiated the authentication request, but the request is not completed. For such methods, server	VERIFICATION_PENDING	Populated with IDs	Contains challenge sets, including the	Populated with result.

attemptResponseCode	Description	attemptReasonCode	context	challengeMethods	credentialValidationResults
	creates and returns a "referenceId" to the client. Any subsequent/verify calls (from client) to poll the status of such method must contain the referenceId in the verify request. If a subsequent/verify call (for a IN_PROCESS method) does not contain the referenceId, the server assumes it is a new request, cancels the current in-process method, and initiates a new one, resulting in a new referenceId. From server to client, the referenceId is set by the server in the method's AuthenticationMethodVersion.referenceId field. From client to server, the client is expected to set the referenceId in the method's Credential.referenceId field.			current IN_PROCESS method with its referenceId.	

Check Status Call

Check Status Request

There are two ways to check the status of the current authentication attempt:

- From the server's response to the previous verify call
- By calling status explicitly

Response to Verify

The server's response to the verification request contains both the validation results of the last call to verify and the remaining challenge methods of the current authentication attempt.

The validation results contain the response code of the current authentication attempt and a list of results of the method verification from the last verify call. Each method verification result contains the method name and the response code that indicates either success or failure.

The remaining challenges are what is required of the client to complete the current authentication attempt. In the case of a successful authentication attempt, there should be no remaining challenge method.

Explicit Call to Check Status

The client can make an explicit call to check the status of the current authentication attempt. The response contains:

- Response code of the current authentication attempt (success or challenge)
- Subject name
- Policy ID
- List of the method names that have successfully completed during this attempt.

Unlike the response to verify, the response of the status call does not include the list of the remaining challenge methods or the list of methods that have failed in the current attempt.

The following graphic illustrates the relationships among the Check Status properties.



Check Status Example

The following example shows how to use Check Status in Java:

```

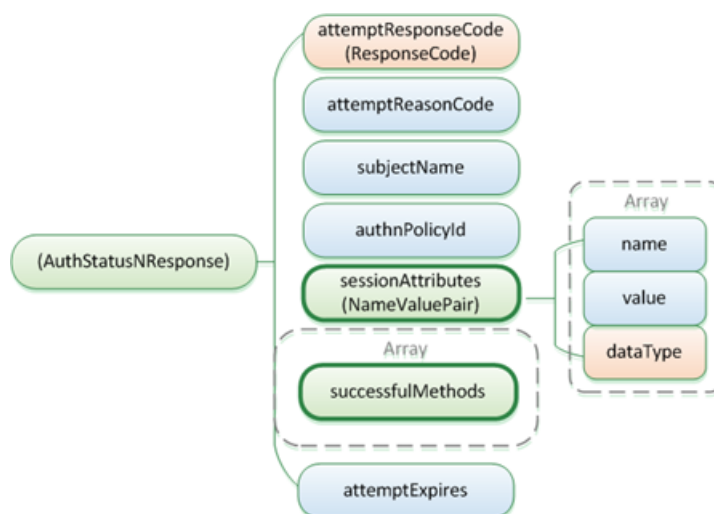
CheckStatus statusRequest = new CheckStatus();
statusRequest.setAuthnAttemptId(attemptId); // the current
authentication attempt id is all it needs
AuthNStatusResponse serverStatusResponse = mfaApi.status
(statusRequest);
  
```

Check Status Parameters

Parameter	Description	Type	Required?
authnAttemptId	The authentication ID provided as a result of an initialize call. This call may have been performed in another client or session.	string	Required
removeAttemptId	Requests to remove the authentication attempt ID as a part of this "check" call. Default is true.	boolean	Optional

Check Status Response

The following graphic illustrates the relationships among the AuthNStatusResponse properties.

**AuthNStatusResponse Properties**

Element	Description	Type
attemptResponseCode	A response status code representation. SUCCESS - The authentication completed successfully. IN_PROCESS - The authentication is not complete but remains in-process. For some out-of-band (PUSH) methods, the client must retry with a verify call. For the Approve method the server receives IN_PROCESS status, and APPROVE_CHECK as method in challengeMethods field. CHALLENGE - The method is incomplete requires method(s) in the challengeMethods. For example, the challengeMethods may contain data requiring the client to perform a secondary challenge. ERROR - A technical error occurred in processing the client request. The authnAttemptId is no longer valid. FAIL - The credentials presented were incorrect. The user	String

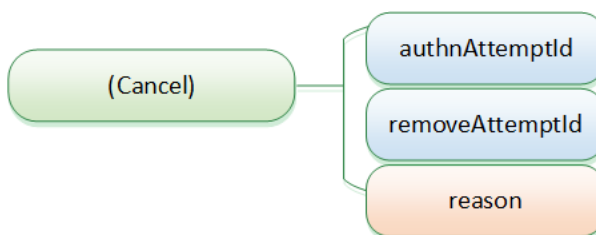
Element	Description	Type
	failed authentication.	
attemptReasonCode	Specific details about the circumstances of result of the action requested.	String
subjectName	Name of the user that completed the authentication.	String
authnPolicyId	Name of the policy ID provided with the initialize call.	String
sessionAttributes	Not supported and is always empty, even if the client had passed sessionAttributes during Initialize.	Array
successfulMethods	An array of methodID strings of methods successfully completed in association with this authentication.	Array of methodID strings.
attemptExpires	Date and time when this authentication attempt expires. This is the local time for the Authentication Manager server or the Cloud Authentication Service together with a time zone offset for UTC time. No verify or status calls can be made after this time (or if the status check requests deletion of the attempt).	String

Cancel Call

Cancel Request

The client may send a Cancel request to cancel an authentication attempt at any point after the initialize request has been sent. The Cancel request must provide the `authnAttemptId` string it received from the initialize response. After a user's authentication attempt has been canceled, all previous authentication results are discarded. The user must restart the authentication process.

The following graphic illustrates the relationships among the Cancel request properties.



Cancel Request Example

The following example shows how a Cancel call provides the `authnAttemptId` to cancel an authentication attempt.

```
{
  "authnAttemptId": "106dd91f-22ab-4eec-92c8-a1a5e01b0da3",
  "reason": "USER_ACTION"
}
```

Cancel Parameters

Parameter	Description	Type	Required?
<code>authnAttemptId</code>	The authentication ID provided as a result of an initialize call. This call may have been performed in another client or session.	String	Required
<code>removeAttemptId</code>	Requests to remove the authentication attempt ID as a part of this "cancel" call. Default is true.	Boolean	Optional
<code>reason</code>	- USER_ACTION: The user intentionally canceled the authentication attempt. - TIME_OUT: The client internally timed out and canceled the attempt. Default is USER_ACTION.	String	Optional

Chapter 3: Sample Clients for the RSA SecurID Authentication API

- Sample Client Overview 40
- Build Sample Clients for RSA Authentication Manager 40

Sample Client Overview

Sample client code for RSA Authentication Manager is available in the **Extras.zip** file in the directory **\RSA SecurID Authentication API**.

This code demonstrates dynamically generating a Java client interface library from the OpenAPI YAML interface definition. It includes two sample programs that use the generated interface to perform authentication.

The first example calls the interface and generates user prompts based on the challenges returned by the server. It continues to prompt the user for challenges until the authentication succeeds or the authentication attempt is halted. This example contains Java examples of generating the HMAC HTTP header value, if that option is enabled at the server. It also configures the SSL client to validate the server's host SSL certificate.

The second example is a simplified example that provides a SecurID tokencode with the Initialize call to perform a single-step SecurID authentication.

Software Requirements

The sample client has the following requirements:

- Java SDK 1.7.0 with unlimited strength crypto extension
- Apache Maven 3.0.4 or later

Maven must be configured to connect to a repository with the **io.swagger:swagger-codegen-maven-plugin** and associated OpenAPI dependencies. The default online repositories contain these components and their dependencies.

You must also change your PATH environment variable so that both the Java SDK and Maven are included.

Files and Directories

The sample client contains the following files and directories:

- **pom.xml**. Top-level Maven build file. Builds the client library and both client samples.
- **README.TXT**. Instructions for compiling and running the sample client.
- **rest-java-client/**. Dynamically constructs a Java client from the schema.
- **rest-test-auth/**. Sample interactive console-based authentication client. This client supports using the Access ID and Access Key.
- **rest-test-cli/**. Sample command-line authentication client.
- **openapi-yaml/rsa-securid-authenticate-api.yaml**. Contains the OpenAPI interface definition source (YAML) file. This contains details on the endpoints and JSON objects. Download the most recent file from RSA Link at <https://community.rsa.com/docs/DOC-71396>.

Build Sample Clients for RSA Authentication Manager

The following procedure generates and compiles the client library, builds the test authentication client, and builds the command line client for two sample clients that call RSA Authentication Manager.

Before you begin

- Meet the software requirements and obtain the necessary files. See [Sample Client Overview on the previous page](#).
- The Authentication Manager Super Admin must do the following:
 - Provide the root certificate.
 - Add a RESTful authentication agent to Authentication Manager
 - Enable the API.
 - Provide the Access Key value.

Procedure

1. Change to the directory where you extracted the OpenAPI REST Interface Source Example.
2. From the top level, run:

```
mvn clean install
```

3. You need to generate or regenerate the Java Client library, **rest-java-client**. Change to the **rest-java-client** directory:

```
cd rest-java-client
```

4. From the top level, run:

```
mvn clean install
```

Note: Warnings, such as "[main] WARN io.swagger.codegen.DefaultCodegen - escape..." can be safely ignored.

The library is located at **target/rest-java-client-version.jar**, where **version** is the RSA Authentication Manager version.

The **pom.xml** file in this directory also generates HTML documentation from the YAML file. This is available in **rest-java-client/target/generated-sources/swagger/index.html**.

The **pom.xml** file also has a commented-out example of how a C++ library can be created.

5. Run the test authentication client, **rest-test-auth**. Type:

```
cd rest-test-auth/target
java -jar rest-test-auth-jar-with-dependencies.jar
```

6. Run the single-step, command-line authentication client, **rest-test-cli**. Type:

```
cd rest-test-cli/target
java -jar rest-test-cli-jar-with-dependencies.jar user passcode accesskey [ server | server-url ]
```

Where:

user. The user ID to be authenticated

passcode. The passcode to use for authentication.

accesskey. The Access Key provided by the server when the REST interface is enabled.

server. (Optional) Server hostname or IP address for default URL.

server-url. (Optional) Authentication service URL.

A property "authenticate.debug" can be set to "true" to enable debugging. For example, type:

```
java -Dauthenticate.debug=true -jar rest-test-cli-jar-with-dependencies.jar ...
```